

4.1 Fundamentals of programming

4.1.1 Programming

4.1.1.1 Data types

Content	Additional information
Understand the concept of a data type.	
Understand and use the following appropriately: • integer • real/float • Boolean • character • string • date/time • pointer/reference • records (or equivalent) • arrays (or equivalent).	Variables declared as a pointer or reference data type are used as stores for memory addresses of objects created at runtime, ie dynamically. Not all languages support explicit pointer types, but students should have an opportunity to understand this data type.
Define and use user-defined data types based on language-defined (built-in) data types.	

4.1.1.2 Programming concepts

Content	Additional information
Use, understand and know how the following statement types can be combined in programs: • variable declaration • constant declaration • assignment • iteration • selection • subroutine (procedure/function).	The three combining principles (sequence, iteration/repetition and selection/choice) are basic to all imperative programming languages.
Use definite and indefinite iteration, including indefinite iteration with the condition(s) at the start or the end of the iterative structure. A theoretical understanding of condition(s) at either end of an iterative structure is required, regardless of whether they are supported by the language being used.	
Use nested selection and nested iteration structures.	
Use meaningful identifier names and know why it is important to use them.	

4.1.1.3 Arithmetic operations in a programming language

Content	Additional information
Be familiar with and be able to use: • addition • subtraction • multiplication • real/float division • integer division, including remainders • exponentiation • rounding • truncation.	

4.1.1.4 Relational operations in a programming language

Content	Additional information
Be familiar with and be able to use: • equal to • not equal to • less than • greater than • less than or equal to • greater than or equal to.	

4.1.1.5 Boolean operations in a programming language

Content	Additional information
Be familiar with and be able to use: • NOT • AND • OR • XOR.	

4.1.1.6 Constants and variables in a programming language

Content	Additional information
Be able to explain the differences between a variable and a constant.	
Be able to explain the advantages of using named constants.	

4.1.1.7 String-handling operations in a programming language

Content	Additional information
Be familiar with and be able to use: • length • position • substring • concatenation • character → character code • character code → character • string conversion operations.	Expected string conversion operations: • string to integer • string to float • integer to string • float to string • date/time to string • string to date/time.

4.1.1.8 Random number generation in a programming language

Content	Additional information
Be familiar with, and be able to use, random number generation.	

4.1.1.9 Exception handling

Content	Additional information
Be familiar with the concept of exception handling.	
Know how to use exception handling in a programming language with which students are familiar.	

4.1.1.10 Subroutines (procedures/functions)

Content	Additional information
Be familiar with subroutines and their uses.	
Know that a subroutine is a named 'out of line' block of code that may be executed (called) by simply writing its name in a program statement.	
Be able to explain the advantages of using subroutines in programs.	

4.1.1.11 Parameters of subroutines

Content	Additional information
Be able to describe the use of parameters to pass data within programs.	
Be able to use subroutines with interfaces.	

4.1.1.12 Returning a value/values from a subroutine

Content	Additional information
Be able to use subroutines that return values to the calling routine.	

4.1.1.13 Local variables in subroutines

Content	Additional information
Know that subroutines may declare their own variables, called local variables, and that local variables: • exist only while the subroutine is executing • are accessible only within the subroutine.	
Be able to use local variables and explain why it is good practice to do so.	

4.1.1.14 Global variables in a programming language

Content	Additional information
Be able to contrast local variables with global variables.	

4.1.1.15 Role of stack frames in subroutine calls

Content	Additional information
Be able to explain how a stack frame is used with subroutine calls to store: • return addresses • parameters • local variables.	

4.1.1.16 Recursive techniques

Content	Additional information
Be familiar with the use of recursive techniques in programming languages (general and base cases and the mechanism for implementation).	
Be able to solve simple problems using recursion.	